

COLLEGE CODE : 8222

COLLEGE NAME : UNIVERSITY COLLEGE OF ENGINEERING, THIRUKKUVALAI

DEPARTMENT : COMPUTER SCIENCE AND ENGINEERING

DATE : 03-10-2025

PROJECT-NAME : WHATNEXT VISION MOTORS

S.NO	NAME	REGISTER NO	NM-ID	PROJECT INVOLMENT
1.	LAKSHMANAN C	822223104036	B17AECA8533B77E882049D64C3CB41D3	MILESTONE-1,2
2.	AKASH K	8222230104008	F7AA36D1F83F1059FD44CE887A9B5EAE	MILESTONE-3,4
3.	SREEJITH S T	822223104030	415F2B8E03559E9BC14493474C01A919	MILESTONE-5,6
4.	NITHISH S P	822223104041	F90CF77B62707869200461FD8356E17C	MILESTONE-7,8

SUBMITTED BY,

LAKSHMANAN C - TEAM LEADER

AKASH K- MEMBER

SREEJITH S T- MEMBER

NITHISH S P - MEMBER

WhatsNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Use Case

WhatsNext Vision Motors is an automotive company that wants to make buying a vehicle easier through technology and automation. To get there, it is rolling out a Salesforce-based solution that simplifies customer ordering, keeps vehicle stock under control, and cuts down on manual work.

The main goal is a smooth, customer-friendly ordering process. When a customer places an order, the system reads the customer's address and suggests the nearest dealer automatically. Customers no longer have to hunt for a dealer themselves, and staff are spared the same effort.

The system also keeps an eye on **vehicle stock** while an order is being placed. If a vehicle is out of stock, the customer simply cannot order it. This prevents wrong orders, spares customers the disappointment, and keeps order processing accurate.

The project also covers **bulk vehicle orders**. A scheduled process regularly checks stock for the ordered vehicles and updates each order's status. If the vehicle is available, the order becomes **Confirmed**. If it is not, the order stays **Pending** until stock arrives.

Overall, the solution is meant to improve **customer satisfaction and operational efficiency**. Automation takes over repetitive admin work, reduces human error, and frees employees to focus on more important tasks. Customers also get clearer order-status information and faster processing.

In short, the WhatsNext Vision Motors Salesforce project brings together **customer management, dealer selection, stock validation, order processing, and automatic status updates** in a single solution, giving the company a more efficient and dependable sales experience.

Requirements

The WhatsNext Vision Motors Salesforce project needs to cover the following areas to improve vehicle ordering and order management.

1. Customer Management

- Users can create and manage customer records.
- Each record holds the customer's name, phone number, email, and address.
- Customer details are available when a vehicle order is created.

2. Vehicle Management

- Vehicle details and current stock quantity are maintained in the system.
- Users can tell at a glance whether a vehicle is available or out of stock.
- Stock figures can be updated whenever needed.

3. Dealer Management

- The system stores each dealer's name, location, address, and contact details.
- It supports multiple dealers across different locations.
- Dealer information is available while customer orders are processed.

4. Nearest Dealer Identification

- When a customer creates an order, the system picks up the customer's location from their address.
- It then finds the nearest available dealer automatically.
- The suggested dealer is linked to the vehicle order.
- Customers and employees no longer have to search for a suitable dealer by hand.

5. Vehicle Order Creation

- Users can create vehicle orders on behalf of customers.
- Each order records the customer, vehicle, quantity, dealer, and order status.
- Vehicle availability is validated before an order is processed.

6. Stock Availability Validation

- Stock is checked before an order is created or confirmed.
- If enough stock is available, the order goes ahead.
- If the vehicle is out of stock, the customer cannot place the order.
- A clear error message explains that the vehicle is unavailable.

7. Order Status Management

The order status depends on whether the vehicle is in stock.

Stock Availability	Order Status
Vehicle available	Confirmed
Vehicle unavailable	Pending

- A **Confirmed** order means the vehicle is available.
- A **Pending** order means the vehicle is currently unavailable and needs further processing.

8. Bulk Order Processing

- The system can process multiple vehicle order records at once.
- Bulk orders are handled without anyone updating each record by hand.
- Stock availability is checked for every vehicle in a bulk order.

9. Scheduled Automation

- A scheduled Salesforce process runs automatically at a set time.
- It finds the vehicle orders that need a status update.
- It checks current stock and updates each order's status.
- Orders for available vehicles become **Confirmed**.
- Orders for unavailable vehicles stay (or become) **Pending**.

10. Automation and Error Reduction

- Repetitive order-processing tasks are automated.
- Manual data entry is kept to a minimum.
- Validation rules, Apex automation, batch processing, and scheduled jobs are used where they fit.
- Meaningful error messages appear when a requirement is not met.

11. Salesforce User Access

- The right Salesforce users can create and manage customers, vehicles, dealers, and vehicle orders.
- Each user only gets access to the records and functions their role needs.
- Permissions are set up for the required objects and fields.

12. Overall Business Requirement

The finished Salesforce solution should give WhatsNext Vision Motors an automated vehicle-ordering process that follows this flow:

Customer → Select Vehicle → Check Stock → Identify Nearest Dealer → Create Order → Confirm/Pending → Automated Status Updates

This will improve **customer convenience, stock accuracy, order processing, and overall operational efficiency.**

Project Scope

1. Salesforce CRM Implementation

- Store and manage vehicle details, stock availability, and dealer information in Salesforce.
- Track customer orders, test drives, and service requests efficiently.
- Automate workflows to assign orders to the nearest dealer based on the customer's location.

2. Process Automation

- Prevent order placement if the vehicle is out of stock.
- Auto-assign orders to the nearest dealer based on the customer's location.
- Send automated email reminders for scheduled test drives.

3. Apex and Trigger

- Implement Apex triggers to enforce business rules such as stock validation and automatic dealer assignment.
- Use trigger handlers to follow best practices and keep the code modular and easy to maintain.

4. Batch Jobs

- Develop a batch Apex job that periodically checks vehicle stock levels and updates availability.
- Send scheduled email notifications for stock replenishment and order processing.

What You'll Learn

1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder

4. Record-Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

Milestone 1: Developer Account Setup

Activity 1: Creating a Developer Account

Create a developer org in Salesforce.

1. Go to <https://developer.salesforce.com/signup>
2. On the sign-up form, enter the following details:

Build enterprise-quality apps fast to bring your ideas to life

- Build apps fast with drag and drop tools
- Customize your data model with clicks
- Go further with Apex code
- Integrate with anything using powerful APIs
- Stay protected with enterprise-grade security
- Customize UI with clicks or any leading-edge web framework

Sign up for your Salesforce Developer Edition
A full-featured copy of the Platform, for free

Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial.

First Name* Last Name*

Email*

Role*

Company*

- a) First name and last name
- b) Email
- c) Role: Developer
- d) Company: College Name
- e) Country: India
- f) Postal code: your pin code
- g) Username: a combination of your name and company

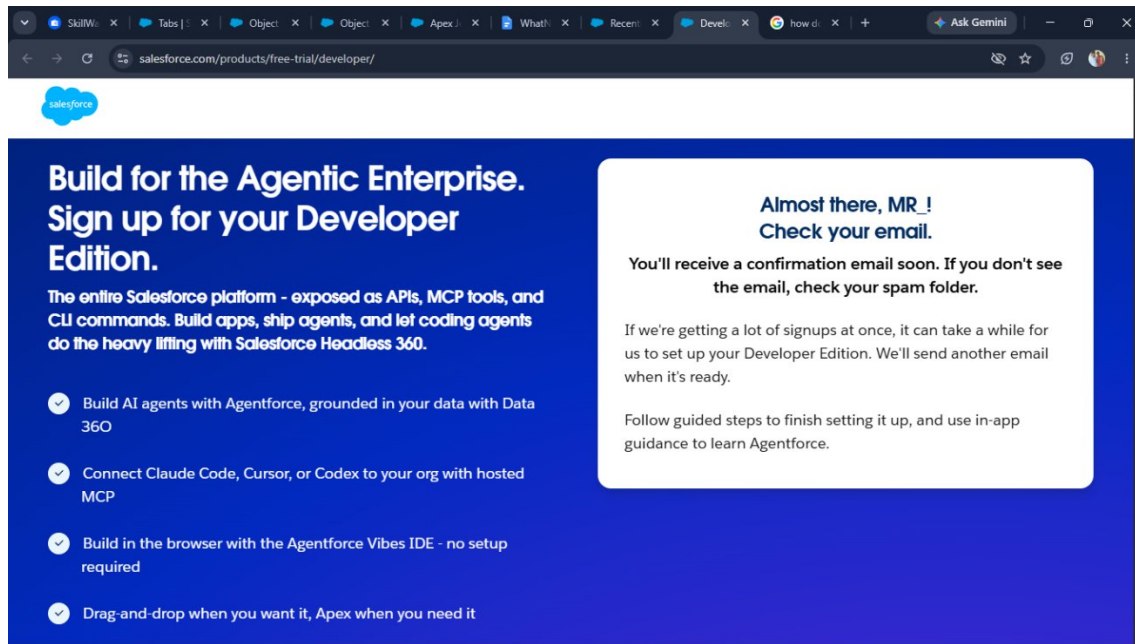
The username does not have to be a real email address. Anything in the format `username@organization.com` will work.

3. Click **Sign me up** once everything is filled in.

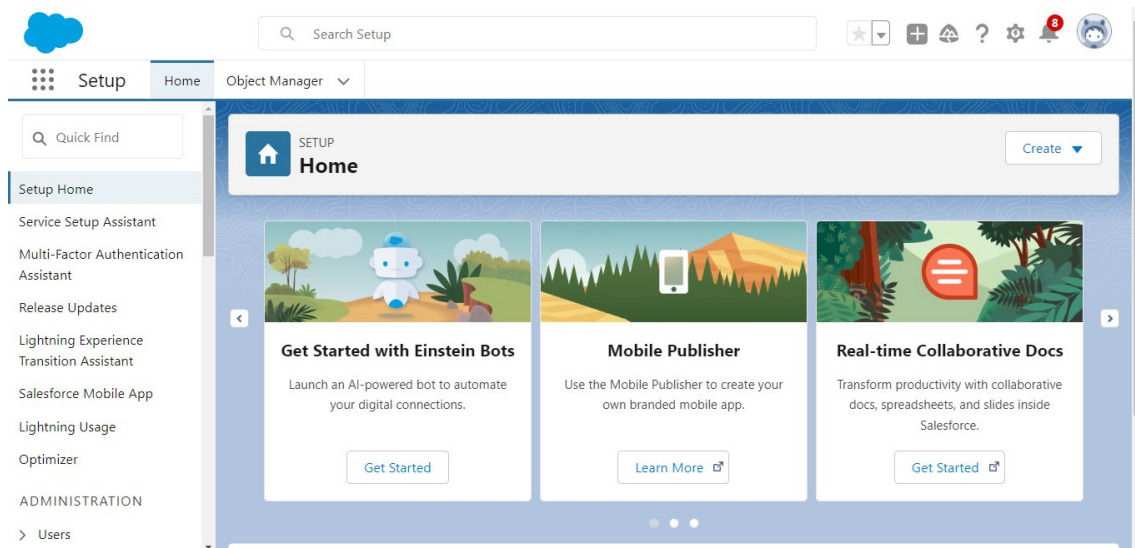
Activity 2: Account Activation

1. Open the inbox of the email address you signed up with and click **Verify Account** in the Salesforce email. It can take 5–10 minutes to arrive.

2. Set a password, answer the security question, and click **Change Password**.



3. You will then be redirected to your Salesforce Setup page.

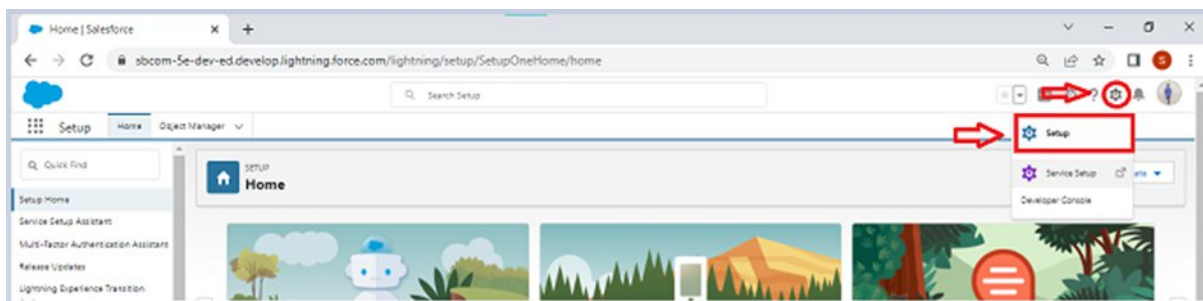


Milestone 2: Objects and Relationships

The project uses the following custom objects.

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To navigate to the Setup page: click the gear icon, then click **Setup**.



Activity 1: Create the Vehicle Object

The Vehicle custom object keeps the details of every vehicle in one place.

To create the object:

1. From the Setup page, click **Object Manager** → **Create** → **Custom Object**.



2. Enter the label name: **Vehicle**
3. Enter the plural label name: **Vehicles**
4. Under Record Name Label and Format, set:
 - Record Name: Vehicle Name

- Data Type: Text
5. Select **Allow Reports**.
 6. Select **Allow Search**, then click **Save**.

Activity 2: Create the Dealer Object

The Dealer custom object stores detailed information about each dealer.

To create the object:

1. From the Setup page, click **Object Manager** → **Create** → **Custom Object**.
2. Enter the label name: **Vehicle Dealer**
3. Enter the plural label name: **Vehicle Dealers**
4. Under Record Name Label and Format, set:
 - Record Name: Dealer Name
 - Data Type: Text
5. Select **Allow Reports**.
6. Select **Allow Search**, then click **Save**.

The screenshot shows the Salesforce Setup page in Developer Edition. The navigation menu on the left includes Setup, Home, and Object Manager. The main content area is titled 'Vehicle' and shows the 'Custom Field Definition Edit' interface. The 'Field Information' section includes fields for Field Label (Dealer), Field Name (Dealer), Description, Help Text, Data Owner (User), Field Usage (--None--), Data Sensitivity Level (--None--), and Compliance Categorization (Available: PII, HIPAA, GDPR, PCI; Chosen:). The 'Lookup Options' section at the bottom shows 'Related To' as Zone and 'Child Relationship Name' as Vehicles.

Milestone 3: Tabs

Activity 1: Creating a Custom Tab (Vehicle)

1. Go to Setup, type **Tabs** in the Quick Find box, click **Tabs**, and under Custom Object Tabs click **New**.

The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with "tab" entered. The main content area is titled "Custom Tabs" and includes a "New" button. Below this is a table of "Custom Object Tabs" with columns for Action, Label, Tab Style, and Description. The table lists several tabs for the "Vehicle" object, including "Vehicle Customers", "Vehicle Dealers", "Vehicle Orders", "Vehicles", "Vehicle Service Requests", and "Vehicle Test Drives". Each tab has a corresponding icon and a "New" button.

Action	Label	Tab Style	Description
Edit Del	Vehicle Customers	Helicopter	
Edit Del	Vehicle Dealers	Laptop	
Edit Del	Vehicle Orders	Cell phone	
Edit Del	Vehicles	Car	
Edit Del	Vehicle Service Requests	IP Phone	
Edit Del	Vehicle Test Drives	Jewel	

2. Select the object (Vehicle) and choose any tab style, then click **Next**. Keep the defaults on the Add to Profiles page and click **Next**, keep the defaults on the Add to Custom App page, and click **Save**.

The screenshot shows the "Custom Tab Definition Edit" page for the "Vehicle Dealers" tab. The page includes a "Custom Object Tab Information" section with fields for Tab Label, Object, and Tab Style. The "Tab Label" is "Vehicle Dealers", the "Object" is "Vehicle Dealer", and the "Tab Style" is "Laptop". There is also a section for "Optional) Choose a Home Page Custom Link to show as a splash page the first time your users click on this tab." with a dropdown menu set to "--None--". A description field is also present.

Field	Value
Tab Label	Vehicle Dealers
Object	Vehicle Dealer
Tab Style	Laptop

Activity 2: Creating Tabs for the Other Objects

Create tabs for the remaining objects you set up earlier in the same way.

Milestone 4: The Lightning App

Activity 1: Create a Lightning App

To create a Lightning app:

1. Go to Setup, search for “App Manager” in Quick Find, select **App Manager**, and click **New Lightning App**.

The screenshot shows the Salesforce Setup interface. In the left sidebar, the 'App Manager' link is highlighted under the 'Apps' section. The main content area is titled 'Lightning Experience App Manager' and displays a table of 32 items. The table columns are: App Name, Developer, Description, Last Modified, App Type, and Visibility. The table lists various Salesforce apps, including Agentforce Grid, Agentforce Studio, All Tabs, Analytics Studio, App Launcher, Approvals, Automation, Bolt Solutions, Community, Content, and Data Cloud. The 'App Manager' link in the sidebar is highlighted, and the 'New Lightning App' button is visible in the top right corner of the main content area.

	App Name	Developer	Description	Last Modified	App Type	Visibility
1	Agentforce Grid	AIWorkbench	Access Agentforce Grid	9/21/2026, 12:57 A...	Lightning	✓
2	Agentforce Studio	AgentforceStudio	Agentforce Studio	9/21/2026, 12:57 A...	Lightning	✓
3	All Tabs	AllTabSet		9/21/2026, 12:50 A...	Classic	✓
4	Analytics Studio	Insights	Build CRM Analytics dashboards and apps	9/21/2026, 12:50 A...	Classic	✓
5	App Launcher	AppLauncher	App Launcher tabs	9/21/2026, 12:50 A...	Classic	✓
6	Approvals	Approvals	Manage approvals and approval flows	9/21/2026, 12:50 A...	Lightning	✓
7	Automation	FlowsApp	Automate business processes and repetitive tasks.	9/21/2026, 12:57 A...	Lightning	✓
8	Bolt Solutions	LightningBolt	Discover and manage business solutions designed for your ind...	9/21/2026, 12:50 A...	Lightning	✓
9	Community	Community	Salesforce CRM Communities	9/21/2026, 12:50 A...	Classic	✓
10	Content	Content	Salesforce CRM Content	9/21/2026, 12:50 A...	Classic	✓
11	Data Cloud	Audience360	Build a thorough and complete understanding of your custome...	9/21/2026, 12:50 A...	Lightning	✓

2. Fill in the app details and branding as follows:

App Name: WhatsNext Vision Motors

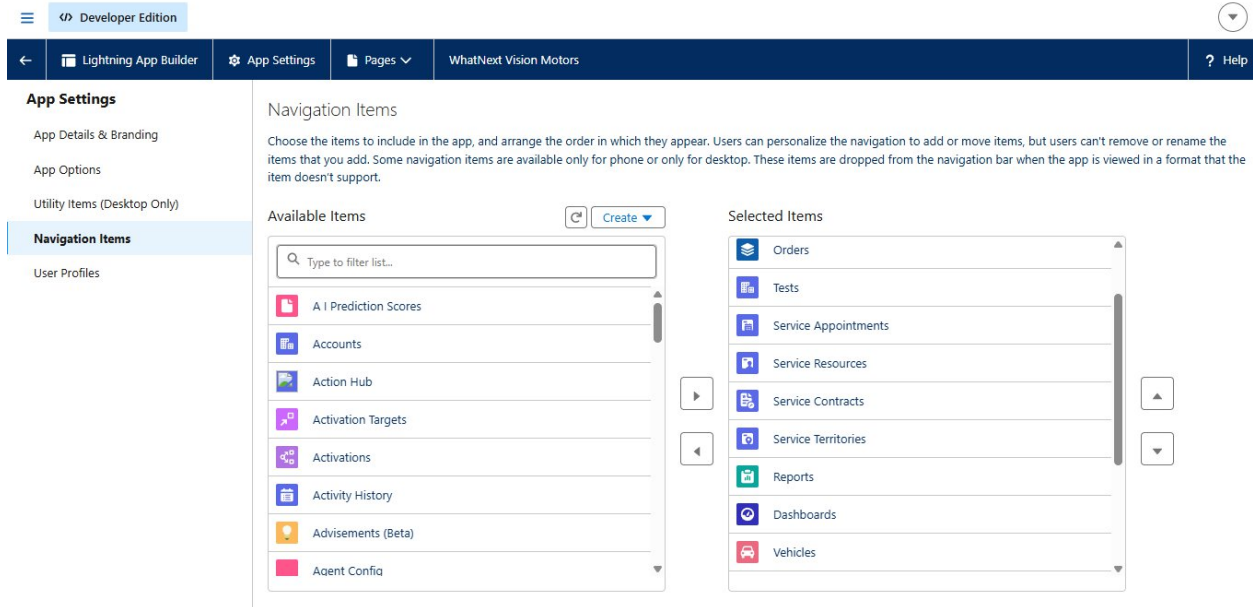
Developer Name: populated automatically

Description: add a short, meaningful description

Image: optional

Primary Color Hex Value: keep the default

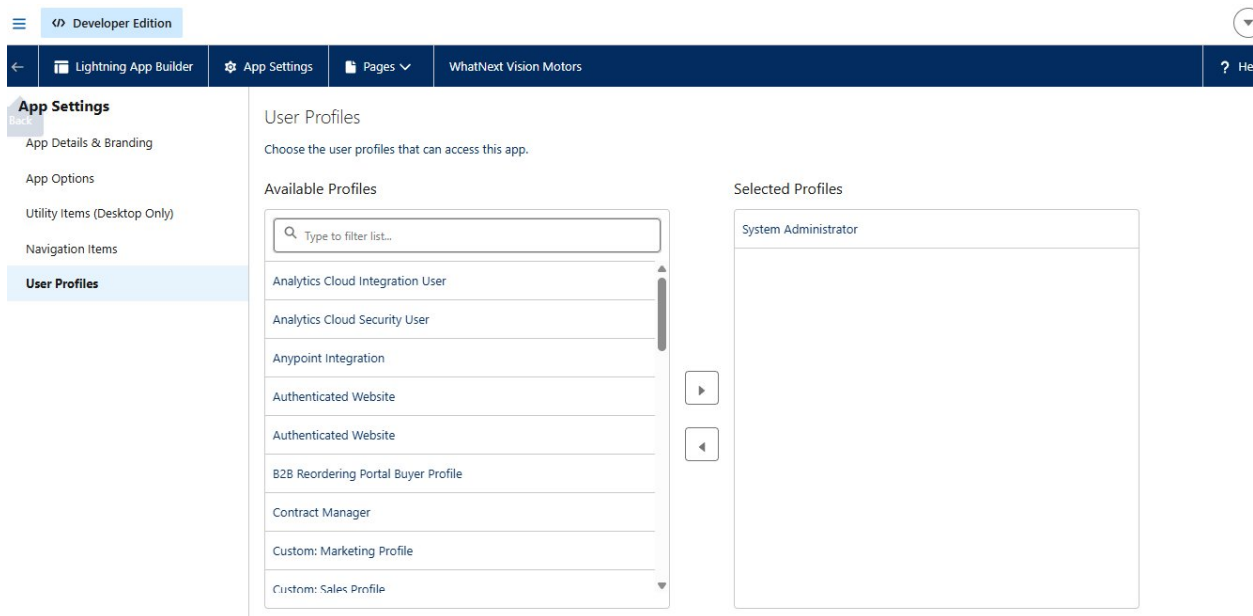
3. Click **Next**. Keep the defaults on the App Options page and click **Next**, then keep the defaults on the Utility Items page and click **Next**.



4. To add navigation items, search for Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, and Dashboard, and move each one across with the arrow button. Then click **Next**.

Note: choose the custom objects you created in the earlier activities.

5. To add user profiles, search for the System Administrator profile, move it across with the arrow button, and click **Save & Finish**.



Milestone 5: Fields and Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle Name** (Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Vehicle_Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2. Vehicle_Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Test_Drive_Date__c** (Date)

- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

Activity 1: Creating a Field in the Vehicle Object

1. Go to Setup, click **Object Manager**, type the object name (Vehicle) in the Quick Find box, and open the object.
2. Click **Fields & Relationships**, then click **New**.
3. Select the data type **Picklist**.

The screenshot shows the Salesforce Setup interface. At the top, there's a 'Developer Edition' label and a search bar labeled 'Search Setup'. Below the navigation bar, the 'Setup' menu is open, and 'Object Manager' is selected. The 'Vehicle' object is selected in the Object Manager. The left sidebar shows the 'Fields & Relationships' tab selected. The main content area shows the 'Data Type' dropdown menu, which is open, displaying various data types with their descriptions:

- None Selected**: Select one of the data types below.
- Auto Number**: A system-generated sequence number that uses a display format you define. The number is automatically incremented for each new record.
- Formula**: A read-only field that derives its value from a formula expression you define. The formula field is updated when any of the source fields change.
- Roll-Up Summary**: A read-only field that displays the sum, minimum, or maximum value of a field in a related list or the record count of all records listed in a related list.
- Lookup Relationship**: Creates a relationship that links this object to another object. The relationship field allows users to click on a lookup icon to select a value from a popup list. The other object is the source of the values in the list.
- Master-Detail Relationship**: Creates a special type of parent-child relationship between this object (the child, or "detail") and another object (the parent, or "master") where:
 - The relationship field is required on all detail records.
 - The ownership and sharing of a detail record are determined by the master record.
 - When a user deletes the master record, all detail records are deleted.
 - You can create rollup summary fields on the master record to summarize the detail records.
 The relationship field allows users to click on a lookup icon to select a value from a popup list. The master object is the source of the values in the list.
- External Lookup Relationship**: Creates a relationship that links this object to an external object whose data is stored outside the Salesforce org.
- Checkbox**: Allows users to select a True (checked) or False (unchecked) value.
- Currency**: Allows users to enter a dollar or other currency amount and automatically formats the field as a currency amount. This can be useful if you export data to Excel or another spreadsheet.
- Date**: Allows users to enter a date or pick a date from a popup calendar.

4. Click Next.

The screenshot shows the Salesforce Setup interface in Developer Edition. The breadcrumb trail is 'SETUP > OBJECT MANAGER'. The page title is 'Vehicle Service Request'. On the left, a sidebar lists various configuration options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, and List View Button Layout. The 'Details' section is selected. The main content area is titled 'Edit Custom Object Vehicle Service Request'. It contains a 'Custom Object Definition Edit' form with buttons for 'Save', 'Save & New', and 'Cancel'. The form has three main sections: 'Custom Object Information', 'Object Classification', and 'Deployment Status'. The 'Custom Object Information' section includes fields for 'Label' (Vehicle Service Request), 'Plural Label' (Vehicle Service Requests), 'Object Name' (Vehicle_Service), and a 'Description' text area. It also has checkboxes for 'Starts with vowel sound' and 'Context-Sensitive Help Setting' (set to 'Open the standard Salesforce.com Help & Training window').

This screenshot shows the 'Object Classification' and 'Deployment Status' sections of the 'Vehicle Service Request' object definition. The 'Object Classification' section includes a list of checkboxes: 'Allow Reports' (checked), 'Allow Activities' (unchecked), 'Track Field History' (unchecked), 'Allow in Chatter Groups' (unchecked), and 'Enable Licensing' (unchecked). Below this, the 'Object Classification' section explains that when these settings are enabled, the object is classified as an Enterprise Application object, and when disabled, it is classified as a Light Application object. The 'Deployment Status' section has two radio buttons: 'In Development' (unchecked) and 'Deployed' (checked). The 'Search Status' section has a checkbox for 'Allow Search' (checked). A 'What is this?' link is present next to the 'Deployment Status' section.

5. Fill in the details as follows:

- Field Label: Vehicle Model
- Picklist Values: enter the values (for example Sedan, SUV, EV)
- Click **Next** → **Next** → **Save & New**

Milestone 6: Flows

Flow 1: Creating a record-triggered flow to assign the nearest dealer based on the customer's location

Step 1: In Quick Find, type **Flows** and click **Flows**. Click **New Flow**.

Step 2: Select **Triggered Automation** from the flow options.

Step 3: Select **Record-Triggered Flow**.

Step 4: Select the **Vehicle Order** object.

Trigger the flow when: a record is created.

Set Entry Condition: All Conditions Are Met (AND)

Field: Status__c

Operator: Equals

Value: Pending

Step 5: Click + and select **Get Records**.

Step 6: Label it **Get Customer Information**.

Object: Vehicle Customer

Condition:

Field: Id

Operator: Equals

Value: {!\$Record.Vehicle_Customer__c}

Step 7: Click + and select **Get Records** again. Label it **Get Nearest Dealer**.

Object: Vehicle Dealer

Condition:

Field: Dealer_Location__c

Operator: Equals

Value: {!Get_Customer_Information.Address__c}

Step 8: Click + and select **Update Records**. Label it **Assign Dealer to Order**.

How to Find Records to Update and Set Their Values: use the IDs and all field values from a record or record collection.

Select Record(s) to Update: {!Get_Nearest_Dealer}

Step 9: Click **Save** and give the flow a label.

Label Name: Auto Assign Dealer

Step 10: Activate the flow.

The screenshot shows the Salesforce Setup interface. On the left, the navigation menu includes 'Setup', 'Home', and 'Object Manager'. A search bar at the top contains 'flows'. The main content area displays the 'Test Drive Reminder' flow configuration. The 'Flow Detail' section shows the flow label 'Test Drive Reminder', description, and various settings. The 'Flow Versions' table at the bottom lists the flow's history.

Flow Detail											
Flow Label	Test Drive Reminder					Flow API Name	Test_Drive_Reminder				
Description						Namespace Prefix					
Environments	Default					Type	Autolaunched Flow				
Active Version	1					URL	/flow/Test_Drive_Reminder				
Trigger	Record—Run After Save					Activated/Deactivated By	LAKSHMANAN CHINNAIYAN, 10/1/2026, 11:28 PM				
Modified By	LAKSHMANAN CHINNAIYAN, 10/1/2026, 11:28 PM					Created By	LAKSHMANAN CHINNAIYAN, 10/1/2026, 11:28 PM				

Flow Versions											
Action	Flow Label	Version	Description	Built with	Created Date	Type	Activation Status	Status	Run in Mode	API Version for Running the Flow	Log Metrics to Data Cloud
Open Run Deactivate	Test Drive Reminder	1		Flow Builder	10/1/2026, 11:28 PM	Autolaunched Flow	Active	Activated	Default Mode	67.0	☐

Flow 2: Creating a record-triggered flow to email the customer a test drive reminder

Step 1: Select **Record-Triggered Flow**.

Step 2: Select the **Vehicle Test Drive** object.

Trigger the flow when: a record is created or updated.

Set Entry Condition: All Conditions Are Met (AND)

Field: Status__c

Operator: Equals

Value: Scheduled

Step 3: Click + **Add Scheduled Paths** below the trigger.

Label: Reminder Before Test Drive

Time Source: Test_Drive_Date__c

Offset Number: 1

Offset Options: Days Before

Click **Done**.

Step 4: Get the customer's details.

1. Click + **Add Element** → **Get Records**.
2. Label: Get Customer Information
3. Object: Vehicle_Customer__c
4. Filter Conditions: Id = {!\$Record.Vehicle_Customer__c}
5. How Many Records to Store: Only the first record
6. How to Store Record Data: Automatically store all fields

Step 5: Send the reminder email.

1. Click + **Add Element** → **Action**.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder
4. Subject: "Reminder: Your Test Drive is Tomorrow!"
5. Recipient Address List: {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body: True
7. Body: Create a variable
8. API Name: EmailSent

Step 6: Click Save.

Label Name: Test Drive Reminder

Step 7: Activate the flow.

Flow page after creation:

The screenshot displays the Salesforce Flow page for 'Auto Assign Dealer'. The page includes a sidebar with navigation options like 'Setup', 'Home', and 'Object Manager'. The main content area shows the flow details, including the flow label, description, API name, and environment. Below the details is a table of flow versions.

Action	Flow Label	Version	Description	Built with	Created Date	Type	Activation Status	Status	Run in Mode	API Version for Running the Flow	Log Metrics to Data Cloud
Open Run Deactivate	Auto Assign Dealer	2		Flow Builder	10/1/2026, 10:56 PM	Autolaunched Flow	Active	Activated	Default Mode	67.0	☐
Open Run Del Activate	Auto Assign Dealer	1		Flow Builder	9/30/2026, 5:56 AM	Autolaunched Flow	Inactive	Draft	Default Mode	67.0	☐

Milestone 7: Apex, Triggers and Batch Jobs

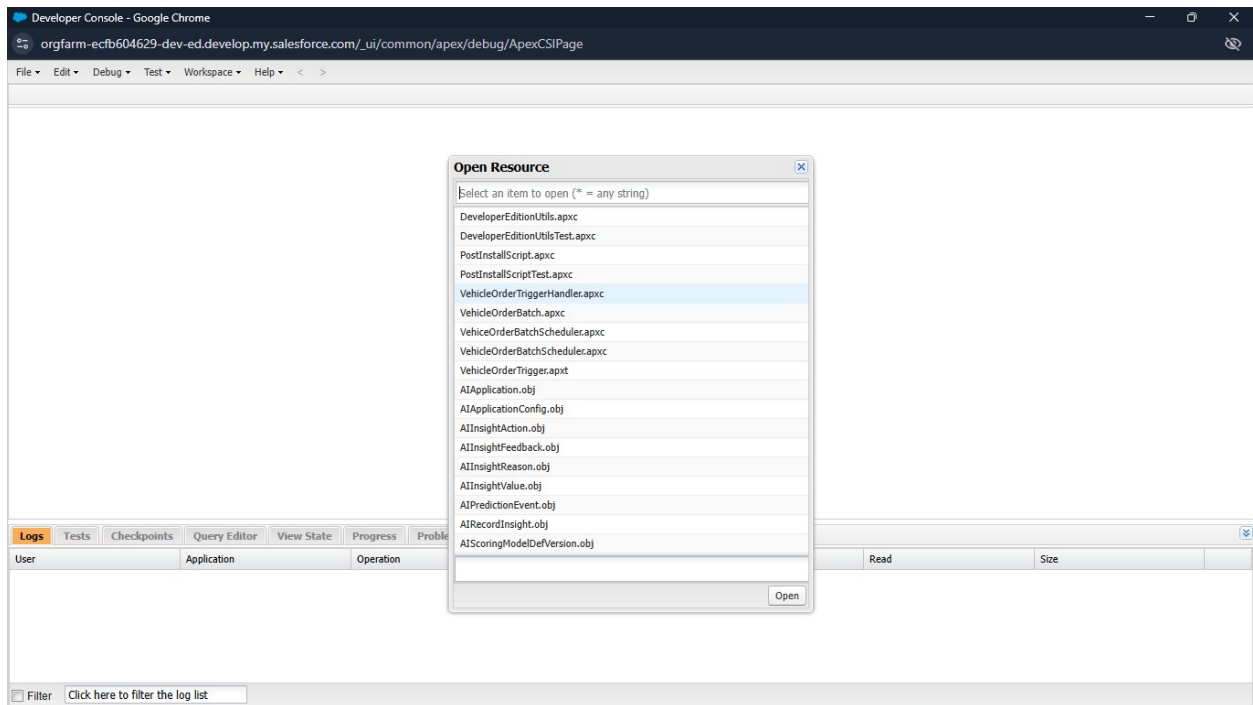
Creating the Apex Trigger and Batch Jobs

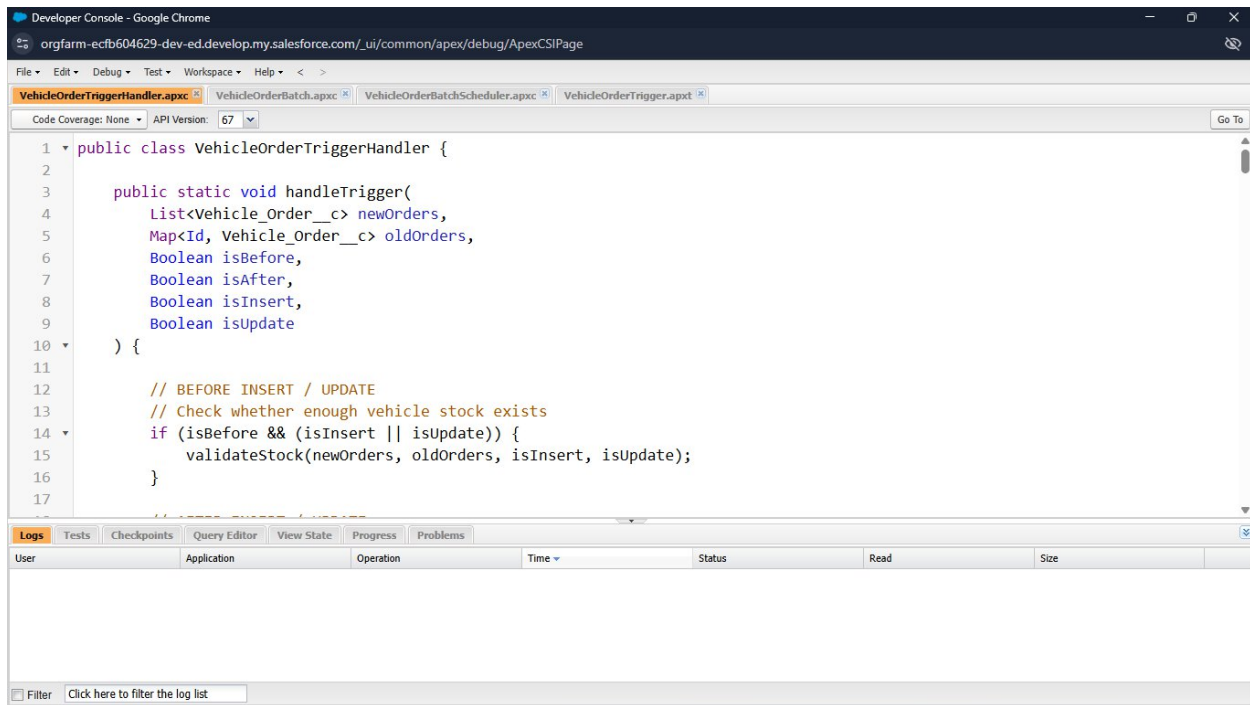
Step 1: Open the **Developer Console** from the gear icon.

Step 2: Click **File** → **New** → **Apex Class**.

Step 3: Name the Apex class **VehicleOrderTriggerHandler**.

Step 4: Write the Apex code.





Source code:

```

public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
        Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter,
        Boolean isInsert, Boolean isUpdate) {

        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
                FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {

```

```

        Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
        if (vehicle.Stock_Quantity__c <= 0) {
            order.addError(
                'This vehicle is out of stock. Order cannot be placed.');
```

```

        }
    }
}

// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();

    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
            FROM Vehicle__c WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c > 0) {
                    vehicle.Stock_Quantity__c -= 1;
                    vehiclesToUpdate.add(vehicle);
                }
            }
        }

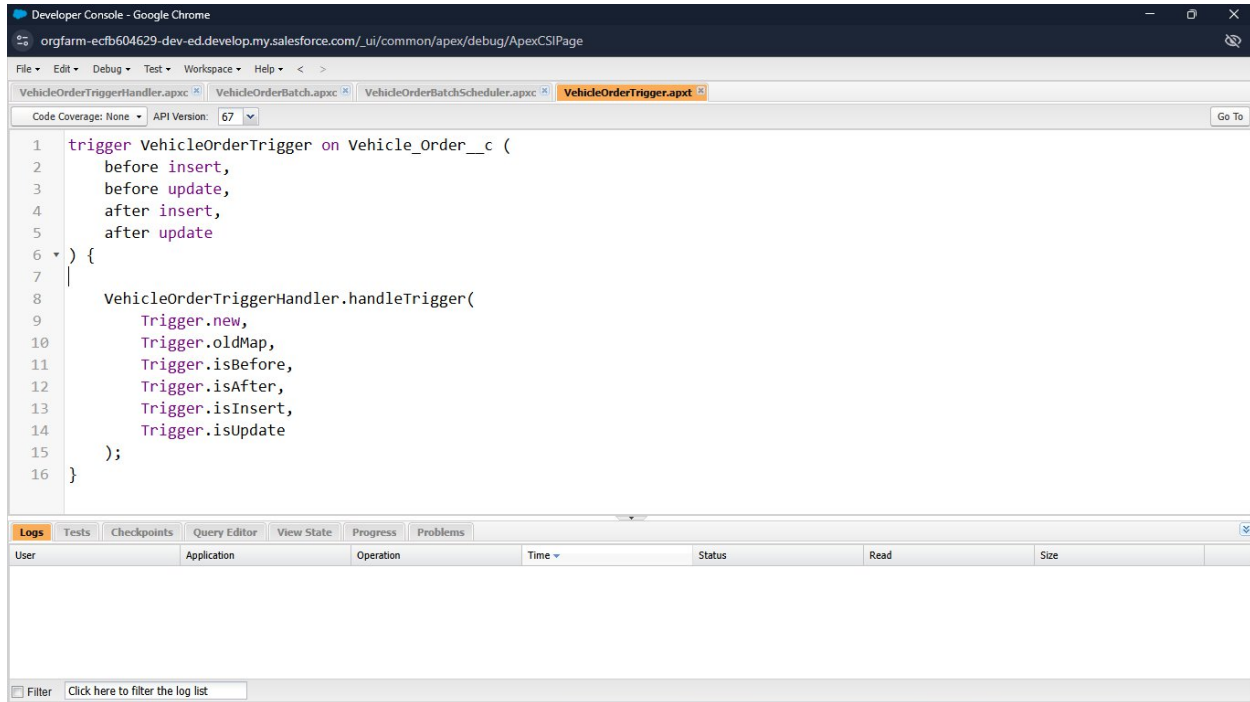
        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }
}
}

```

Step 5: Create the Apex trigger (**File** → **New** → **Apex Trigger**).

Step 6: Enter a name for the trigger and select the **Vehicle Order** object (Vehicle_Order__c).

Step 7: Call the Apex class from the trigger.



The screenshot shows the Salesforce Developer Console with the Apex trigger code for Vehicle_Order__c. The code is as follows:

```
1 trigger VehicleOrderTrigger on Vehicle_Order__c (  
2     before insert,  
3     before update,  
4     after insert,  
5     after update  
6 ) {  
7  
8     VehicleOrderTriggerHandler.handleTrigger(  
9         Trigger.new,  
10        Trigger.oldMap,  
11        Trigger.isBefore,  
12        Trigger.isAfter,  
13        Trigger.isInsert,  
14        Trigger.isUpdate  
15    );  
16 }
```

The console also shows the 'Logs' tab at the bottom, which is currently empty. The 'Filter' button is visible at the bottom left of the logs section.

Step 8: Create the batch job.

Scenario: a customer places an order, but the vehicle is out of stock.

- The order stays **Pending**.
- Once new stock is added, the batch job updates the order to **Confirmed**.

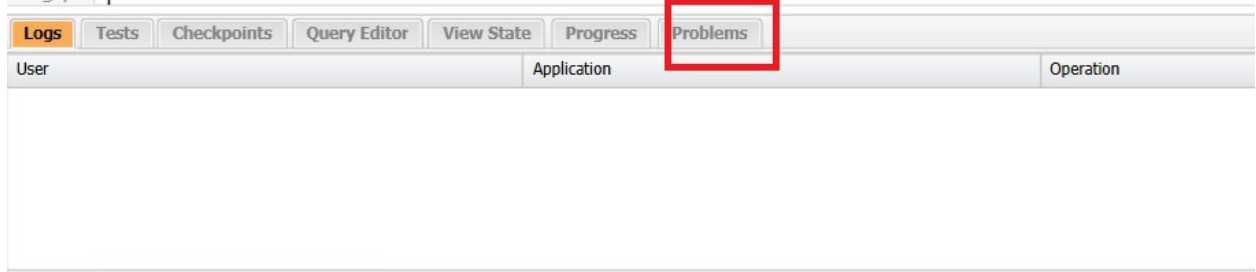
Code Coverage: None API Version: 62

```

1  global class VehicleOrderBatch implements Database.Batchable<sObject> {
2
3
4  global Database.QueryLocator start(Database.BatchableContext bc) {
5      return Database.getQueryLocator([
6          SELECT Id, Status__c, Vehicle__c
7          FROM Vehicle_Order__c
8          WHERE Status__c = 'Pending'
9      ]);
10 }
11
12 global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
13     Set<Id> vehicleIds = new Set<Id>();
14
15     for (Vehicle_Order__c order : orderList) {
16         if (order.Vehicle__c != null) {
17             vehicleIds.add(order.Vehicle__c);
18         }
19     }
20
21     if (!vehicleIds.isEmpty()) {
22         Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
23         for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
24             vehicleStockMap.put(vehicle.Id, vehicle);
25         }
26
27         List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
28         List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
29
30         for (Vehicle_Order__c order : orderList) {
31             if (vehicleStockMap.containsKey(order.Vehicle__c)) {

```

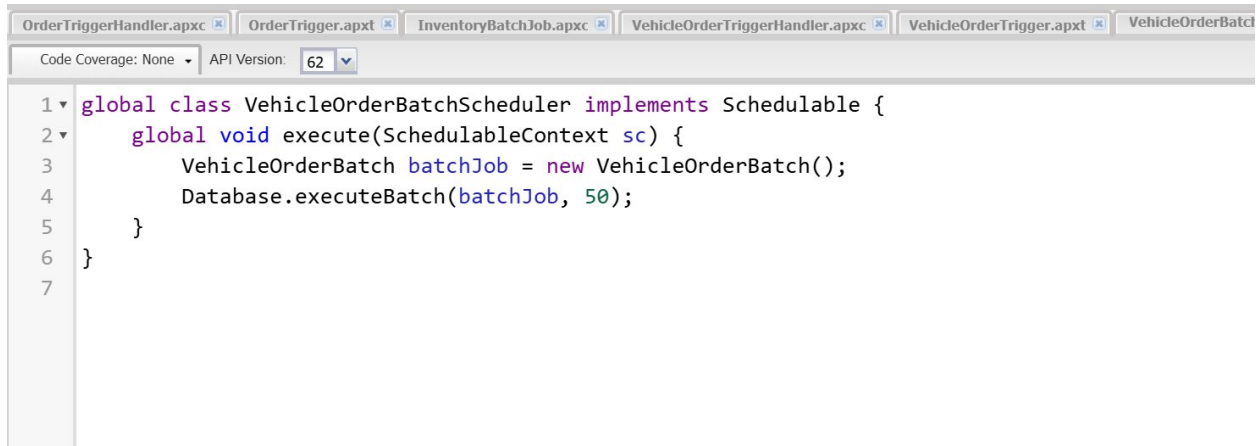
```

30     if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31         Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
32         if (vehicle.Stock_Quantity__c > 0) {
33             order.Status__c = 'Confirmed';
34             vehicle.Stock_Quantity__c -= 1;
35             ordersToUpdate.add(order);
36             vehiclesToUpdate.add(vehicle);
37         }
38     }
39 }
40
41 if (!ordersToUpdate.isEmpty()) {
42     update ordersToUpdate;
43 }
44
45 if (!vehiclesToUpdate.isEmpty()) {
46     update vehiclesToUpdate;
47 }
48 }
49 }
50
51 global void finish(Database.BatchableContext bc) {
52     System.debug('Vehicle order batch job completed.');


The screenshot shows the Salesforce IDE interface. The top part is the Apex code editor with the code from the previous block. Below the editor is a toolbar with tabs: Logs, Tests, Checkpoints, Query Editor, View State, Progress, and Problems. The Problems tab is highlighted with a red rectangular box. Below the toolbar is a table with columns: User, Application, and Operation. The table is currently empty.


```

Step 9: Create a schedule class that calls the batch class.



The screenshot shows the Salesforce IDE interface with several tabs open: OrderTriggerHandler.apxc, OrderTrigger.apxt, InventoryBatchJob.apxc, VehicleOrderTriggerHandler.apxc, VehicleOrderTrigger.apxt, and VehicleOrderBatchJob.apxc. The VehicleOrderBatchJob.apxc tab is active. The code editor shows the following Apex code:

```

1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50);
5     }
6 }
7

```

Source code:

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Step 10: Schedule the batch job to run daily at 12:00 PM (noon).

```
Enter Apex Code
1 String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
2 System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
3
```

☐ Open Log

Source code:

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp,
    new VehicleOrderBatchScheduler());
```

Step 11: Check that the scheduled job is running.

orgfarm-ecfb604629-dev-ed.develop.my.salesforce-setup.com/lightning/setup/AsyncApexJobs/home

Developer Edition

Search Setup

Setup Home Object Manager

jo

Environments

Jobs

Apex Flex Queue

Apex Jobs

Background Jobs

Bulk Data Load Jobs

Scheduled Jobs

Didn't find what you're looking for? Try using Global Search.

Click here to go to the new batch jobs page

Apex Jobs

Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.

Percent of Asynchronous Apex Used: 0%
You have currently used 0 asynchronous Apex operations out of an allowed 24-hour organization limit of 250,000. To learn about how this limit is calculated and what contributes to it, see the Lightning Platform Apex Limits topic.

View: All Create New View

Action	Submitted Date	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class	Apex Method	Apex Job ID
	10/2/2026, 12:51 AM	Scheduled Apex	Queued		0	0	0	CHINNAIYAN LAKSHMANAN		VehicleOrderBatchScheduler		707gK0000wOrFT

My Service Journey

Thank you